Hier-FUN: Hierarchical Federated Learning and Unlearning in Heterogeneous Edge Computing

Zhenguo Ma, Huaqing Tu, Li Zhou, Pengli Ji, Xiaoran Yan, *Member, IEEE*, Hongli Xu, *Member, IEEE*, Zhiyuan Wang, and Suo Chen

***Abstract*—Federated learning (FL) has emerged as a pivotal paradigm for distributed model training in edge computing (EC), enabling cooperation among numerous Internet of Things devices while safeguarding their data privacy. Despite its successes in machine learning, concerns regarding data security and model fidelity necessitate the efficient unlearning of target device, i.e., federated unlearning (FUN). However, due to resource constraints, device heterogeneity, and non-independent and identically distributed (Non-IID) data, securely eliminating a device's impact without retraining the model from scratch presents a complex challenge. In response to these challenges, we propose a hierarchical FUN framework, called Hier-FUN. Hier-FUN organizes edge devices into *K* clusters, each managed by a head device responsible for aggregating local models within the cluster. To expedite both the learning and unlearning processes of Hier-FUN, we design a heuristic algorithm to determine an appropriate value for *K* based on devices' data distributions and available resources. In addition, Hier-FUN denies the communication between the server and cluster heads during training, which can constrain the influence sphere of target device and accelerate the unlearning process. We conduct extensive experiments using real-world datasets, and the experimental results illustrate that Hier-FUN can improve test accuracy by 3.19% during the learning phase and achieve a 6.8× speedup during unlearning compared with the baseline methods.**

***Index Terms*—Device heterogeneity, edge computing (EC), federated unlearning (FUN), hierarchical architectures, non-independent and identically distributed (Non-IID).**

I. Introduction

With the fast proliferation of Internet of Things (IoT) devices, federated learning (FL) [1] has become a promising paradigm for orchestrating massive IoT devices to collaboratively train a unified global model within the scenarios of edge computing (EC) [2]. By transmitting trained models instead of raw data between the central server and edge devices, FL inherently safeguards data privacy and significantly mitigates communication overhead [3], [4], [5], [6], [7], [8]. As a consequence, edge learning for beyond 5G (B5G) networks has garnered widespread adoption recently [9], [10], [11], [12], [13], [14], [15].

The existing FL methods primarily concentrate on expediting the learning trajectory of global model, as evidenced by recent works, such as [16], [17], [18], [19], and [20]. However, amidst these efforts, there is a growing tendency toward unlearning the data residing on edge devices. For example, both the general data protection regulation (GDPR) [21] from the European Union and the California Consumer Privacy Act (CCPA) [22] from the United States emphasize the right (of data) to be forgotten. Additionally, within the context of an FL system, the integrity of the data on participating devices is not always guaranteed. Some devices may harbor malicious intent, while others may unwittingly possess outdated or low-quality data [23]. As a consequence, within a parameter-server (PS) [24]-based FL system, it becomes crucial to endow the server with the capability to eliminate the contribution from specific devices, which can protect the data security and ensure the model fidelity. To this end, recent research efforts have honed in on federated unlearning (FUN) [23], [25], [26], [27], [28], [29], expecting to efficiently unlearn a device without retraining the model from scratch, as demonstrated in Fig. 1.

In the pursuit of efficient FUN in EC, we encounter several critical challenges that extend beyond the inherent stochasticity and incrementality characteristics of model training.

1) *Constrained Resources:* EC is inherently characterized by the universal and persistent constraints of computational and communication resources on edge devices [17], [18], [19], [20]. However, the introduction of FL and FUN amplifies the resource requirements due to the periodic updates of local models and the frequent model transmissions between the server and devices. This clash with the constrained resources of edge devices exacerbates the challenge, necessitating innovative strategies to mitigate resource overhead while maintaining performance.
2) *Heterogeneous Devices:* The devices engaged in FL and FUN encompass a diverse array of edge devices, ranging

Received 23 August 2024; revised 12 November 2024; accepted 15 November 2024. Date of publication 20 November 2024; date of current version 26 March 2025. This work was supported in part by the National Key Research and Development Program of China under Grant 2022YFB4501504; in part by the National Science Foundation of China (NSFC) under Grant 72101068; and in part by the China Postdoctoral Science Foundation under Grant 2024M75986. *(Corresponding authors: Huaqing Tu; Pengli Ji.)*

Zhenguo Ma, Li Zhou, Pengli Ji, and Xiaoran Yan are with the Research Center for Data Hub and Security, Zhejiang Laboratory, Hangzhou 311121, Zhejiang, China (e-mail: zgma@zhejianglab.org; zhou.li@zhejianglab.org; jipengli@zhejianglab.org; yanxr@zhejianglab.org).

Huaqing Tu is with the Research Center for Innovation of Intelligent Computing Facilities, Zhejiang Laboratory, Hangzhou 311121, Zhejiang, China (e-mail: tuhuaqing@zhejianglab.org).

Hongli Xu, Zhiyuan Wang, and Suo Chen are with the School of Computer Science and Technology, University of Science and Technology of China, Hefei 230027, Anhui, China, and also with the Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou 215123, Jiangsu, China (e-mail: xuhongli@ustc.edu.cn; cswangzy@mail.ustc.edu.cn; chensuo@mail.ustc.edu.cn).

Digital Object Identifier 10.1109/JIOT.2024.3502666

2327-4662 © 2024 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

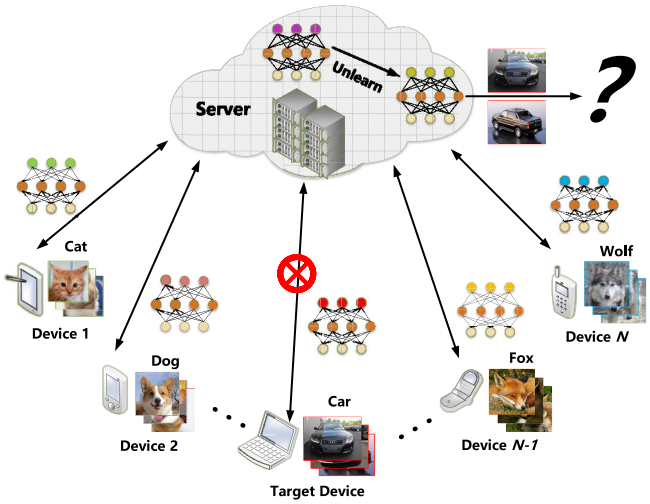


Fig. 1. Illustration of FUN.

from smartphones to vehicles, each characterized by distinct computational capacities, data sizes, and network connectivity profiles [30], [31], [32], [33], [34]. The heterogeneity among edge devices leads to significant variability in the time required for local model updates and model transmission. As such, devising adaptive mechanisms to accommodate this diversity while ensuring efficient collaboration poses a formidable challenge.

3) *Non-IID Data:* Another fundamental challenge arises from the non-identical and independently distributed (Non-IID) nature of data generated by edge devices [35], [36], [37], [38]. Concretely, users may come from diverse organizations or communities, and contribute data with unique characteristics and distributions, deviating from the conventional IID assumption. This discrepancy complicates the aggregation of knowledge from Non-IID data, undermining the performance of FUN when unlearning the data from specific devices.

A straightforward approach to ensure that the global model completely eliminates the contribution of a particular device involves retraining the model from scratch. Nevertheless, such a strategy often incurs prohibitively expensive computational overhead, leading to a significant strain on the constrained resources of the involved devices. Therefore, existing methods explore alternative approaches to approximate FUN by generating an approximation of the fully retrained model. One notable method in this realm is FedEraser, which is specifically designed to efficiently achieve FUN by reconstructing the unlearned model leveraging historical model updates [39]. Despite its efficiency, FedEraser introduces considerable memory overhead for the server in storing gradient replicas. To mitigate this challenge, Golatkar et al. [40] performed a corrective Newton step on the trained model using the retained data $\mathcal{D}$-$\mathcal{D}_i$, which requires the global access to overall dataset $\mathcal{D}$. Furthermore, in the context of convolutional neural networks (CNNs), Wang et al. [25] presented a method focused on the class discrimination of different channels, wherein the most discriminative channels are pruned to facilitate the unlearning of the target class. However, it's worth noting that this approach is confined to removing samples from specific classes and is tailored exclusively for the models with CNN architecture.

The aforementioned methods aim to accelerate the unlearning process, overlooking the adverse effect of device heterogeneity on model training. Instead, the hierarchical architectures that divide all devices into several clusters and aggregate the model updates within the confines of each cluster solely can help to tackle the heterogeneous devices. Besides, by canceling the transmission between the server and cluster heads, the influence sphere of the target device to be unlearned is constrained, and the unlearning process can be expedited. In this context, Su and Li [28] introduced an efficient FUN approach, called KNOT, which organizes the devices with similar training time and data distribution into the same cluster. Nonetheless, considering the Non-IID data, performing clustered aggregation among the devices with similar data distribution will definitely harm the generalization performance of the trained model. Furthermore, KNOT performs clustering on devices with a prespecified number of clusters, necessitating substantial prior knowledge.

Motivated by this observation, we introduce a novel framework, termed Hier-FUN, which leverages hierarchical architecture to achieve efficient learning and unlearning in EC. Concretely, Hier-Fun organizes all the edge devices into K clusters based on their possessed resources and data distributions. However, the performance of Hier-FUN mainly relies on the choice of cluster number (denoted as K). Notably, when devices are divided into more clusters (large K), the unlearning of target device can be quickly achieved, but the training performance may be degraded on Non-IID data. In contrast, the training performance is enhanced if small K is adopted, but the unlearning process becomes time-consuming. Hence, determining an appropriate value for K remains a critical challenge for Hier-FUN. In addition, regarding the Non-IID data, it is also challenging for Hier-FUN to cluster edge devices while ensuring the model performance. Our primary contributions are summarized as follows.

1) We propose a novel framework for FUN, which can accelerate both the learning and unlearning processes utilizing the hierarchical architectures.
2) We design a heuristics algorithm to determine an appropriate value for K, and perform clustering on edge devices based on their possessed resources and data distributions, so as to tackle the aforementioned challenges.
3) We conduct extensive experiments on three real-world datasets. The experimental results demonstrate that Hier-FUN significantly improves test accuracy by 3.19% during the learning phase and achieves a 6.8× speedup during unlearning, when compared to the baseline methods.

The remainder of this article is organized as follows. Section II provides the preliminaries of hierarchical FL (HFL) and its convergence upper bound. We present the proposed framework and detailed algorithm design in Section III. In Section IV, the experiments are conducted, and the results are presented and analyzed. We summarize the previous related works in Section V. Finally, the conclusion of this article is shown in Section VI.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

II. Preliminaries and Problem Formulation

A. Hierarchical Federated Learning

Under the typical FL framework [1], a central server orchestrates massive edge devices to train a globally shared model, so as to solve various training tasks without exposing the private local data of devices. Formally, we consider an FL system that is comprised of a central server and N edge devices. Assume that N edge devices are divided into K clusters, i.e., $\{\mathcal{C}_1, \mathcal{C}_2, \ldots, \mathcal{C}_K\}$, with $n_k = |\mathcal{C}_k|$ devices in each cluster $\mathcal{C}_k$. Subsequently, upon the established cluster, HFL is performed with the following three steps, i.e., *local updating*, *edge aggregation*, and *global aggregation*.

Local Updating: For each device $i \in [N]$, a local dataset $\mathcal{D}_i = \{(x_j, y_j)_{j\in[d_i]}\}$ that follows the device-specific distribution P_i is stored and maintained, where $[N] = \{1, 2, \ldots, N\}$ denotes the set of indices for all devices, and d_i represents the number of samples in dataset $\mathcal{D}_i$. Based on the dataset $\mathcal{D}_i$, the local loss function f_i of device i is expressed as follows [26]:

$$f_i\left(w_i^t\right) = \frac{1}{d_i}\sum_{j=1}^{d_i} F_i\left(w_i^t; (x_j, y_j)\right) \tag{1}$$

where w_i^t represents the local model of device i at round t and $F_i(w_i^t; (x_j, y_j))$ denotes the loss function on jth sample of device i. Generally, gradient descent-based methods, such as [32] and [41], are widely adopted in each local update (i.e., iteration) to minimize (1). We note that more than one iteration is performed in local updating, so as to achieve a tradeoff between communication and computation efficiency. At the τth iteration of round t, device i updates the local model $w_i^{t,\tau}$ as follows:

$$w_i^{t,\tau} = w_i^{t,\tau-1} - \eta\nabla f_i\left(w_i^{t,\tau-1}\right) \tag{2}$$

where η denotes the learning rate of model training and $\nabla f_i(w_i^{t,\tau-1})$ indicates the gradient of local loss function on local model $w_i^{t,\tau-1}$. After H local iterations, the local model $w_i^{t,H}$ is transmitted to the cluster head k for edge aggregation. The important notations are listed in Table I.

Edge Aggregation: The edge aggregation on cluster head $k \in [K]$ is triggered upon receiving the updated local models from all the devices in cluster $\mathcal{C}_k$. Following the standard federated averaging (FedAvg) algorithm [1], the edge aggregation on cluster head k can be expressed as follows:

$$w_k^t = \sum_{i\in\mathcal{C}_k}\frac{d_i w_i^{t,H}}{D_k} = w_k^{t-1} - \eta\sum_{i\in\mathcal{C}_k}\frac{d_i\nabla f_i\left(w_k^{t-1}\right)}{D_k} \tag{3}$$

where w_k^t represents the aggregated model on cluster head k at round t and $D_k = \sum_{i\in\mathcal{C}_k} d_i$ is the accumulated number of samples on all devices in cluster $\mathcal{C}_k$.

Global Aggregation: The central server usually updates the global model in an asynchronous manner [19], so as to tackle the heterogeneous devices. Concretely, as soon as one aggregated model from arbitrary cluster head k is received, the global aggregation at the tth round is performed as follows:

$$w^t = (1-\alpha)w^{t-1} + \alpha w_k \tag{4}$$

TABLE I
Table of Main Notations

Notations	Semantics
N	number of participating devices
K	number of divided clusters
f	global loss function
η	learning rate during model training
w	global model
w^*	optimal model
$\overline{w}$	unlearned model
T	number of global rounds
H	number of local iterations
$\mathcal{M}$	correlation matrix
$\mathcal{C}_k$	set of devices in cluster k
D_k	number of samples on all devices in cluster $\mathcal{C}_k$
w_k	aggregated model in cluster $\mathcal{C}_k$
$\mathcal{D}_i$	local dataset of device i
d_i	number of samples in dataset $\mathcal{D}_i$
f_i	local loss function of device i
∇f_i	gradient of loss function on device i
w_i	local model of device i
$\mathcal{T}_i$	consumed time of device i

where w^t denotes the global model at round t and α illustrates the weight of cluster model w_k.

B. Problem Formulation

In this article, we aim to accelerate both the learning and unlearning process in EC by solving the problem of device clustering. Due to the constrained and heterogeneous resources of edge devices, the time consumed by different devices to complete a round varies a lot. Hence, we let $\mathcal{T}_i$ denote the time consumed by device i to complete a round, and the maximum time among the devices in cluster $\mathcal{C}_k$ is represented by $\mathcal{T}_k$. Similarly, regarding the Non-IID data, the deviation of cluster $\mathcal{C}_k$ can be measured by $\nabla\widetilde{f_k} = \nabla f_k - \nabla f$, where $\nabla f_k = (1/|\mathcal{C}_k|)\sum_{i\in\mathcal{C}_k}\nabla f_i$ and $\nabla f = (1/N)\sum_{i=1}^N \nabla f_i$ denote the aggregated gradient in cluster $\mathcal{C}_k$ and the global gradient, respectively. Thus, according to [19] and [28], the problem of device clustering can be formulated as follows.

$$\min_{p_i^k}\sum_{i=1}^N\sum_{k=1}^K p_i^k\cdot\mathrm{dist}(i,\mathcal{C}_k)$$

$$\begin{cases}\textbf{s.t. } \mathrm{dist}(i,\mathcal{C}_k) = \varphi_1\cdot|\mathcal{T}_k - \mathcal{T}_i| + \varphi_2\cdot\cos(\nabla f_i, \nabla\widetilde{f_k}) \quad \forall i,k & \text{(5a)}\\ \sum_{k=1}^K n_k = N \qquad \forall k & \text{(5b)}\\ n_k \in \left\{\left\lfloor\frac{N}{K}\right\rfloor, \left\lceil\frac{N}{K}\right\rceil\right\} \qquad \forall k & \text{(5c)}\\ \sum_{k=1}^K p_i^k = 1 \qquad \forall i,k & \text{(5d)}\\ p_i^k \in \{0,1\} \qquad \forall i,k & \text{(5e)}\end{cases}$$

Due to devices' heterogeneous resources, the fast devices are required to wait for the slow devices, incurring nonnegligible waiting time [32]. In addition, the Non-IID data leads

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

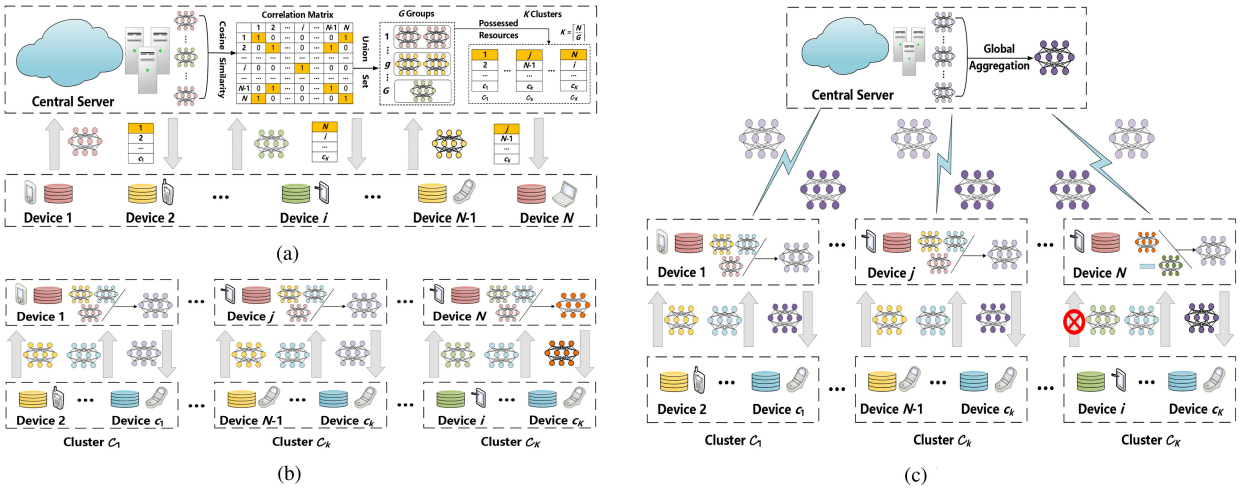


Fig. 2. Illustration of training process of Hier-FUN. (a) Federated clustering. (b) In-cluster learning. (c) Unlearning and fine-tune.

to the deviation of local models from the global model [35]. To accelerate both the learning and unlearning process, the objective of device clustering problem is to jointly minimize $\text{dist}(i, \mathcal{C}_k)$ for all devices i and clusters $\mathcal{C}_k$, where $\text{dist}(i, \mathcal{C}_k) = \varphi_1 \cdot |\mathcal{T}_k - \mathcal{T}_i| + \varphi_2 \cdot \cos(\nabla f_i, \nabla \widetilde{f}_k)$ is the weighted summation of waiting time and gradient deviation, φ_1 and φ_2 represent the discount factors, and cos denotes the cosine similarity between two vectors. Equations (5b) and (5c) demonstrate that the well-known balanced clustering strategy is used to improve the convergence performance of the model. Equation (5d) ensures that each device i can exactly be assigned to one cluster, where device i is (not) included in cluster $\mathcal{C}_k$ if $p_i^k = 1$ ($p_i^k = 0$). By minimizing the objective in (5), the devices with similar hardware specifications and complementary data distributions will be assigned to the same cluster, and the model performance can be guaranteed.

III. Proposed Framework—Hier-FUN

It is worth noting that the performance of Hier-FUN is closely related to the number of clusters K. Consequently, in order to achieve a tradeoff between training performance and unlearning efficiency, we first determine the cluster number (i.e., K) using a heuristic method, and then allocate all edge devices into K clusters based on their available resources and data distributions. As shown by Fig. 2, the framework of Hier-FUN mainly consists of three phases, i.e., *federated clustering*, *in-cluster learning*, as well as *unlearning and fine-tune*.

A. Federated Clustering

The algorithm for federated clustering of edge devices is outlined in Algorithm 1. First, the central server broadcasts the initialized global model w^0 to all devices. After retrieving the global model, device i performs H local iterations and calculates the accumulated gradient $\nabla f_i(w^0)$. Then, the gradient $\nabla f_i(w^0)$ and the consumed time $\mathcal{T}_i$ are uploaded to the server. Based on the received gradient, the server calculates the correlation matrix $\mathcal{M} \in \{0, 1\}^{N \times N}$ among devices, shown by line 3 of Algorithm 1. For sake of clarity, let $\mathcal{M}_{i,j}$ denote the element in the ith row and jth column of $\mathcal{M}$. The value of $\mathcal{M}_{i,j}$ for devices i and j is set as 1 if the cosine similarity of their gradients is larger than a constant δ. Based on the union set algorithm [42], we let each device be an independent set by specifying its parent as itself. Subsequently, we merge devices i and j if $\mathcal{M}_{i,j} = 1$, as indicated by line 5 of Algorithm 1. In this way, we can obtain G disjoint groups by dividing the devices with identical parent into the same group. Note that the devices in the same group follow the similar data distribution. However, performing clustered model training with devices from similar data distribution will cause the deviation of cluster model. To get rid of this issue, we aim to divide the devices with complementary data into the same cluster. Given the number of devices N and the number of groups G, the number of clusters that can derive uniform cluster models should be around (N/G). Regarding that increasing the values of K can restrict the influence sphere of the target device, and the unlearning efficiency is improved. We set the number of clusters K as $K = \lceil (N/G) \rceil$ to achieve a tradeoff between learning performance and unlearning efficiency.

To solve the problem in (5) and divide devices into different clusters, we introduce the bipartite graph [43], [44], since only two types of entities (i.e., device and cluster head) are involved. However, the traditional bipartite graph that consists of K cluster heads and N devices can hardly satisfy the constraint in (5c). As a consequence, we establish the bipartite graph based on the relationship between K and N.

1) When N is divisible by K, we repeat each cluster head $k \in [K]$ by (N/K) times.
2) When N cannot be evenly divided by K, we repeat the cluster heads $k \in [1, N \mod K]$ by $\lceil (N/K) \rceil$ times, and the cluster heads $k \in [N \mod K + 1, K]$ are duplicated by $\lfloor (N/K) \rfloor$ times.

In these two cases, total N vertexes are generated on the cluster head side of the bipartite graph. On the client side, we also generate N vertexes for N edge devices. We note that establishing the optimal cluster structure is equivalent to finding the maximum matching of the constructed bipartite graph [43]. Specifically, we first select K arbitrary devices as

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 Federated Clustering

Procedure at the server

1: Send the initialized global model w^0 to all edge devices;
2: Receive $\nabla f_i(w^0)$ and $\mathcal{T}_i$ from each device $i \in [N]$;
3: Calculate the correlation matrix $\mathcal{M} \in \{0,1\}^{N\times N}$, where $\mathcal{M}_{i,j} = 1$ if $\cos(\nabla f_i(w^0), \nabla f_j(w^0)) > \delta$;
4: Set the parent of each device as itself;
5: Merge devices i and j if $\mathcal{M}_{i,j} = 1$, and set the parent of device j's parent to be the parent of device i;
6: Divide the devices with identical parent into the same group, and obtain G disjoint groups;
7: Calculate the number of clusters as $K = \lceil \frac{N}{G} \rceil$;
8: Initialize the centroid for each cluster $\mathcal{C}_k$;
9: **repeat**
10: **for** $i = 1$ to N **do**
11: **for** $k = 1$ to K **do**
12: Calculate $dist(i, \mathcal{C}_k)$ according to Eq. (5a);
13: **end for**
14: **end for**
15: Fill the distance matrix $\mathcal{J}$ with $dist(i, \mathcal{C}_k)$;
16: Divide the devices into different clusters based on $\mathcal{J}$ using the Hungarian algorithm;
17: **for** $k = 1$ to K **do**
18: Update the centroid of cluster $\mathcal{C}_k$ as $\mathcal{T}_k = \max_i(p_i^k \cdot \mathcal{T}_i)$ and $\nabla \widetilde{f_k} = \frac{p_i^k \cdot \nabla f_i}{\sum_i p_i^k} - \nabla f$;
19: **end for**
20: **until** The determined cluster assignment is stable or the objective function in Eq. (5) is not decreased;
21: Send the divided clusters to all devices;

Procedure at device i

1: Receive the global model w^0 from the central server;
2: Perform H local iterations, and calculate the accumulated gradient $\nabla f_i(w^0)$;
3: Upload $\nabla f_i(w^0)$ and consumed time $\mathcal{T}_i$ to the server;
4: Receive the divided clusters from the server;

Algorithm 2 In-Cluster Learning

Procedure at cluster head k

1: Send the initialized model w_k^0 to the devices in $\mathcal{C}_k$;
2: **for** $t = 1$ to T **do**
3: Receive the local gradient $\nabla f_i(w_k^{t-1})$ from device i;
4: Update the accumulated gradient $\nabla \bar{f}_i$ for device i as $\nabla \bar{f}_i = \nabla \bar{f}_i + \frac{d_i}{D_k} \nabla f_i(w_k^{t-1})$;
5: Update the cluster model w_k^t according to Eq. (3);
6: Send the updated model w_k^t to the devices in $\mathcal{C}_k$;
7: **end for**

Procedure at device i

1: Receive the initialized model w_k^0 from cluster head;
2: **for** $t = 1$ to T **do**
3: Perform H local iterations, and calculate the accumulated gradient $\nabla f_i(w_k^{t-1})$;
4: Send $\nabla f_i(w_k^{t-1})$ to cluster head;
5: Receive the updated model w_k^t from cluster head;
6: **end for**

the cluster heads and initialize the centroid for each cluster (line 8 of Algorithm 1). Then, we create a distance matrix $\mathcal{J} \in \mathbb{R}^{N\times N}$ according to the bipartite graph, and $\mathcal{J}_{i,j}$ is calculated as $\mathcal{J}_{i,j} = \text{dist}(i, \mathcal{C}_k)$ $(k = (j-1) \mod K + 1)$, as indicated by lines 10–14 of Algorithm 1. Based on $\mathcal{J}$, we divide the devices into different clusters using the Hungarian algorithm [45].

1) *Step 1:* For each row i in matrix $\mathcal{J}$, we find its minimum value and subtract it from every element within row i.
2) *Step 2:* For each column j in matrix $\mathcal{J}$, we find its minimum value and subtract it from every element within column j.
3) *Step 3:* We try to cover all zeros in matrix $\mathcal{J}$ using the minimum number of lines (either horizontal or vertical). If the total number of lines employed equals N, the Hungarian algorithm terminates, indicating that an optimal assignment has been determined.
4) *Step 4:* After the previous step, we find the minimum uncovered value and subtract it from all uncovered elements. For the elements that are covered twice, we add the minimum uncovered value to them.

The last two steps are iteratively repeated until the condition of N covering lines is satisfied. Subsequently, the assignment between devices and cluster heads can be determined by analyzing the distribution of zeros. However, since the initial cluster heads are randomly selected, the determined cluster assignment may not be optimal. Therefore, after determining the cluster assignment, we update the centroid of each cluster, as illustrated by lines 17–19 of Algorithm 1. Then we iteratively recalculate the distance matrix $\mathcal{J}$ based on the new centroid and conduct the Hungarian algorithm, until the cluster assignment is stable or the objective function in (5) is not decreased. Finally, the server distributes the determined cluster assignment to all devices.

B. In-Cluster Learning

In each cluster $\mathcal{C}_k$, the device with the strongest hardware specifications (i.e., the minimum time $\mathcal{T}_i$) is designated as the cluster head, and is responsible for edge aggregation. Concretely, the initialized model w_k^0 is distributed to all devices in cluster $\mathcal{C}_k$, by line 1 of Algorithm 2. At the device side, the accumulated gradient $\nabla f_i(w_k^{t-1})$ is obtained by performing H local iterations on local data. Then, the gradient $\nabla f_i(w_k^{t-1})$ is uploaded to the cluster head for aggregation. By line 4 of Algorithm 2, cluster head k stores the historical gradient $\nabla \bar{f}_i$ of device $i \in \mathcal{C}_k$ for further processing in *unlearning and fine-tune*. Subsequently, the cluster model w_k^t is updated according to (2), and w_k^t is sent back to edge devices for local updating. Different from the typical HFL framework in Section II-A, Hier-FUN cancels the global aggregation, which can restrict the influence sphere of the target device and accelerate the unlearning process.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

Algorithm 3 Unlearning and Fine-Tune

Input: Pre-trained model w_k^T, target device l
Output: Unlearned model $\overline{w}$
Initialize: $t = 1$

Procedure at the server

1: **repeat**
2: Receive the cluster model $\overline{w}_k^{t-1}$ from head k;
3: Update the unlearned model $\overline{w}^t$ by Eq. (4);
4: Send the updated model $\overline{w}^t$ to cluster head k;
5: Set $t = t + 1$;
6: **until** $f(\overline{w}^t) \leq \xi$
7: Set $\overline{w} = \overline{w}^t$ and send *STOP* flag to each cluster head;

Procedure at cluster head k

1: **if** Target device $l \in \mathcal{C}_k$ **then**
2: Calculate $\overline{w}_k^0 = w_k^T + \eta \nabla \bar{f}_l$;
3: **else**
4: Set $\overline{w}_k^0 = w_k^T$;
5: **end if**
6: **while** *STOP* flag is not received **do**
7: Send model $\overline{w}_k^{t-1}$ to the device $i \in \mathcal{C}_k$, $i \neq l$;
8: Receive the gradient $\nabla f_i(\overline{w}_k^{t-1})$ from device i;
9: Update the unlearned model $\overline{w}_k^t$;
10: Send $\overline{w}_k^t$ to the central server;
11: Receive $\overline{w}^t$ from server and set $\overline{w}_k^t = \overline{w}^t$;
12: Set $t = t + 1$;
13: **end while**
14: Send *STOP* flag to the devices within the cluster;

Procedure at device i

1: **while** *STOP* flag is not received **do**
2: Receive $\overline{w}_k^{t-1}$ from cluster head k;
3: Perform H local iterations, and calculate the accumulated gradient $\nabla f_i(\overline{w}_k^{t-1})$;
4: Send $\nabla f_i(\overline{w}_k^{t-1})$ to cluster head;
5: Set $t = t + 1$;
6: **end while**

C. Unlearning and Fine-Tune

When an unlearning request emerges, the *in-cluster training* is terminated and Hier-FUN enters the *unlearning and fine-tune* phase. Concretely, for the cluster $\mathcal{C}_k$ that consists of target device l, we derive the unlearned model as $\overline{w}_k^0 = w_k^T + \eta \nabla \bar{f}_l$ simply based on the gradient ascent algorithm [46]. Note that the proposed framework is not restricted to gradient ascent, and can be integrated with other unlearning algorithms for further improvement. For the cluster $\mathcal{C}_k$ excluding target device l, the unlearned model is initialized as the pretrained model $\overline{w}_k^0 = w_k^T$, as illustrated by Lines 1–4 of the procedure at cluster head k in Algorithm 3. Then, cluster head k distributes the unlearned model $\overline{w}_k^{t-1}$ to the devices in $\mathcal{C}_k$ (except for l). Based on the received model $\overline{w}_k^{t-1}$, device i calculates the gradient $\nabla f_i(\overline{w}_k^{t-1})$ and uploads to the cluster head. When receiving gradients from all devices in $\mathcal{C}_k$, cluster head k updates the unlearned model $\overline{w}_k^t$ using the received gradients. The updated model $\overline{w}_k^t$ is then transmitted to the server to derive the global unlearned model $\overline{w}^t$. The central server broadcasts $\overline{w}^t$ to cluster head k to start the next round of fine-tuning. The above process is repeated until the global loss function $f(\overline{w}^t)$ is lower than a constant ξ, then the server sets the fine-tuned model as $\overline{w} = \overline{w}^t$ and sends *STOP* flag to all cluster heads.

D. Analysis

1) Convergence Results: Before presenting the convergence results, we first make some assumptions on the loss functions and gradients.

Assumption 1 (β-Smooth): The global loss function f is β-smooth with $\beta > 0$, i.e., it always holds $f(w_2) - f(w_1) \leq \langle \nabla f(w_1), w_2 - w_1 \rangle + (\beta/2)\|w_2 - w_1\|^2$ for any two model parameters w_1 and w_2, where $f(w) = \sum_{i=1}^N (d_i f_i(w)/D)$ and $D = \sum_{i=1}^N d_i$.

Assumption 2 (μ-Strongly Convex): The global loss function f is μ-strongly convex with $\mu > 0$, i.e., it always holds $f(w_2) - f(w_1) \geq \langle \nabla f(w_1), w_2 - w_1 \rangle + (\mu/2)\|w_2 - w_1\|^2$ for any two model parameters w_1 and w_2.

Assumption 3 (Bounded Gradient): There exist two positive constants Q_1 and Q_2 such that for any device $i \in [N]$ and model parameter w, $\mathbb{E}\|\nabla f_i(w; \mathcal{D}_i)\|^2 \leq Q_1$ and $\mathbb{E}\|\nabla f_i(w; \mathcal{D}_i) - \nabla f(w)\|^2 \leq Q_2$.

Based on Assumptions 1–3, we can obtain the convergence bound of the global loss function f as follows.

Theorem 1: Under the above assumptions, when $\eta < (1/\beta)$, the convergence bound of the global loss function f after T rounds can be expressed as

$$\begin{aligned}\mathbb{E}\left[f\left(w^T\right) - f\left(w^*\right)\right] &\leq \left[\frac{K-1}{N} + \alpha(1-\eta\mu)^H\right]^T \left(f(w^0) - f(w^*)\right) \\ &\quad + \frac{(Q_1+Q_2)\left(1 - \left[\frac{K-1}{N} + \alpha(1-\eta\mu)^H\right]^T\right)}{2\eta\mu^2} \end{aligned} \tag{6}$$

where w^0 denotes the initial global model and w^* represents the optimal model that minimizes the loss function f.

Since the similar proof can be found in previous work [19], the proof of Theorem 1 is omitted in this article.

2) Storage Complexity Analysis: To demonstrate the effectiveness of the proposed framework, we compare the storage complexity of Hier-FUN with that of the baselines at each round. For ease of notation, let P denote the parameter size of global model. In terms of FedAvg, the server aggregates the local models (gradients) from all the devices at each round. Hence, at round t, the storage complexity of FedAvg is $\mathcal{O}(N \cdot P)$, where N is the number of devices. For FedEraser, since the server stores the gradients of all the devices from all previous rounds, its storage complexity reaches $\mathcal{O}(t \cdot N \cdot P)$. As for the hierarchical framework (i.e., Hier-FUN and KNOT), by dividing all the devices into disjoint clusters and storing the accumulated gradients of previous rounds, the storage complexity is reduced to $\mathcal{O}(\max([N/K], K) \cdot P)$ (K is the number of clusters), where $(N/K) \cdot P$ and $K \cdot P$ denote the memory overhead for the cluster head and the server, respectively. Even in special cases where $K = 1$ or $K = N$, Hier-FUN can achieve comparable storage complexity with FedAvg, demonstrating the efficacy of the proposed framework.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

3) Optimality Analysis: In this section, we provide the sketch optimality analysis for Algorithm 1, and the detailed proof can be found in [45].

According to the constructed bipartite graph, we can formulate a dual problem for (5).

$$\begin{aligned} \max \quad & u_1 + u_2 + \cdots + u_N + v_1 + v_2 + \cdots + v_N \\ \text{s.t.} \quad & u_i + v_j \leq \mathcal{J}_{i,j} \quad \forall i, j \end{aligned}$$

In other words, we assign u_i and v_j to device i and cluster head $(j-1) \mod K+1$, aiming to maximize their summation while satisfying $u_i + v_j \leq \mathcal{J}_{i,j}$. It is obvious that solution of dual problem is less than the matching value of (5). Hence, we can solve (5) if we find the solution for dual problem while satisfying the constraint. Then, we outline the operations performed at each step of the Hungarian algorithm. It can be readily observed that steps 1 and 2 effectively assign initial values to each u_i and v_j of the dual problem. The reductions applied to the rows and columns correspond equivalently to the corresponding additions to u_i and v_j. We denote u_1 to u_r as the rows selected in step 3, v_1 to v_s as the columns selected in step 3, and e as the minimum value of uncovered elements. The procedures in step 4 essentially accomplish the following:

$$\begin{cases} u_1 = u_1, \ldots, u_r = u_r \\ u_{r+1} = u_{r+1} + e, \ldots, u_N = u_N + e \\ v_1 = v_1 - e, \ldots, v_s = v_s - e \\ v_{s+1} = v_{s+1}, \ldots, v_N = v_N \end{cases}$$

In more straightforward terms, the unselected rows are reduced by the minimum value, while the corresponding u_i of the selected columns is increased by the same amount. Conversely, the selected columns are increased by the minimum value, with the corresponding v_j decreased by that amount. In this way, we have 1) each uncovered element is decreased; 2) each element covered once remains unchanged; and 3) each element covered twice is increased. Hence, the constraint $u_i + v_j \leq \mathcal{J}_{i,j}$ is still satisfied. In addition, since the totally selected rows and columns are less than N, it follows that the number of unselected rows must exceed that of the selected columns, and the objective function in the dual problem is increased. Consequently, if adequate iterations are conducted, the algorithm can find the optimal solution.

E. Discussion

In EC, the participants in FL are usually involved in various kinds of mobile devices. These devices typically connect to base stations (BSs) through unstable wireless links. Consequently, due to the dynamic nature of the EC environment and the mobility of devices, communication timeout may arise as a result of BS handover or unstable network condition. Such timeout can increase the time required for the model to reach convergence, leading to the performance degradation of the model within the allocated time budget. To address this issue, we propose adapting the timeout retransmission mechanism from the transmission control protocol (TCP) to FL. Specifically, we suggest implementing a retransmission mechanism for trained models if no reply is received from the cluster head (or server) within the retransmission timeout (RTO). Additionally, since network condition in EC environments may vary with time, the RTO (denoted as $\mathcal{T}_{\text{RTO}}$) should be adjusted according to real-time network condition. As outlined in [47], $\mathcal{T}_{\text{RTO}}$ can be calculated as follows:

$$\mathcal{T}_{\text{RTO}} = \widehat{\mathcal{T}}_{\text{RTT}} + \varphi * \sigma(\mathcal{T}_{\text{RTT}}) \tag{8}$$

where φ is a factor reflecting network condition and is typically set to 4 [47], $\widehat{\mathcal{T}}_{\text{RTT}}$ represents the smoothed round-trip time by incorporating the historical round-trip time $\mathcal{T}_{\text{RTT}}$, and $\sigma(\mathcal{T}_{\text{RTT}})$ denotes the round-trip time variation.

IV. Experimentation and Evaluation

A. Experimental Setup

Implementation: In this section, we conduct comprehensive experiments to evaluate the performance of our proposed framework. Concretely, we establish an FL experimental environment utilizing the computational power of an AMAX deep learning workstation[1] (CPU: Intel Skylake Xeon, GPU: NVIDIA TESLA V100S 32 GB). Under the simulation environment, we implement Hier-FUN and baselines with the commonly used PyTorch library [48]. Subsequently, we create 22 threads on the workstation, one of which is regarded as the central server. The remaining 21 threads serve as edge devices, including the target device to be unlearned. Regarding the device heterogeneity, we start by independently capturing and quantifying the computation capacity of four distinct categories of commercial devices, namely, smartphones, laptops, Jetson TX2,[2] and Raspberry Pi, in advance. Then, we specify the computation capacity of each simulated device as an average value derived from one specific category of commercial devices.

Datasets: We extensively evaluate the performance of Hier-FUN through experiments conducted on three real-world datasets: 1) Fashion-MNIST [49] (FMNIST for short); 2) CIFAR-10[3]; and 3) CIFAR-100[3]. To represent the Non-IID setting, we randomly split each dataset into 6 partitions with different data distributions. Each partition generates data by selecting samples with a subset of labels as its main labels. The last partition is directly assigned to the target device (usually the last device). As for the remaining partitions, we introduce a factor p representing the percentage of samples from main labels, defining three Non-IID levels: 1) Level 1 ($p = 0.9$); 2) Level 2 ($p = 0.8$); and 3) Level 3 ($p = 0.6$). For example, at Level 2 ($p = 0.8$), each partition contains 80% of samples from main labels, while 20% are evenly drawn from the other main labels. Each of these partitions is further divided into four subsets to serve as local datasets for the remaining 20 edge devices (except for the target device). It is worth noting that the experiments are usually conducted under Level 1, unless otherwise stated.

Models: We evaluate the performance of three different models with distinct architectures over the mentioned real-world datasets: 1) multilayer perceptron (MLP) over

[1] www.amax.com/products/ai-hpc-ml/.
[2] https://developer.nvidia.com/embedded/jetson-tx2-developer-kit.
[3] http://www.cs.toronto.edu/ kriz/cifar.html.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

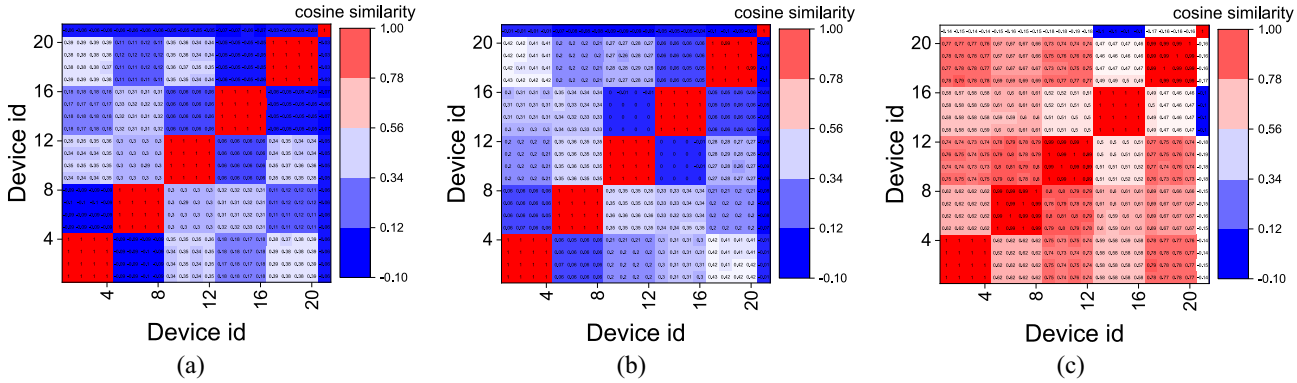


Fig. 3. Cosine similarity of gradients from any two devices under three Non-IID levels. (a) Level 1. (b) Level 2. (c) Level 3.

FMNIST; 2) AlexNet over CIFAR-10; and 3) ResNet-18 over CIFAR-100. First, for the simple FMNIST dataset, we implement a MLP model that consists of five layers with units of 2000, 1000, 500, 100, and 10, respectively [50]. Second, over the CIFAR-10 dataset, we employ the AlexNet model [51], which contains five convolutional layers and three fully connected layers. Lastly, for the CIFAR-100 dataset that is more complex due to more categories, we utilize the well-known ResNet-18 model [52].

Baselines: In order to evaluate the effectiveness of Hier-FUN, we introduce the other six methods as baselines.

1) *FedAvg [1]:* FedAvg updates the global model by aggregating the local models contributed by all edge devices.
2) *ReTrain:* ReTrain is the approach that unlearns the target device by retraining the global model with the retained data.
3) *FedCH [19]:* FedCH aims to assign the devices with similar hardware specifications into the same cluster.
4) *KNOT [28]:* KNOT performs clustering to divide the devices with similar data distribution into the same cluster.
5) *Random:* Random is a variant of KNOT and trains three models with randomly divided clusters.
6) *FedEraser [39]:* FedEraser stores and calibrates the historical gradients on the server.

It is noteworthy that FedCH aims to accelerate the learning process with cluster construction. Therefore, we only evaluate the learning performance (i.e., test accuracy) of FedCH. Regarding the number of partitions (except for the data of target device), the number of clusters K for Random and KNOT is configured as 5, while Hier-FUN determines the value of K via Algorithm 1. Additionally, to handle the poor network condition, we add a baseline, referred to as Hier-FUN-T, which integrates the proposed Hier-FUN framework with timeout retransmission mechanism. To emulate poor network condition, we assume that during each round of the communication process, there is a probability of 0.1 that the local models on edge devices will be dropped.

Parameter Settings: In our experiments, we standardize the batch size and learning rate settings as follows: 64 and 0.0001 for FMNIST, 32 and 0.0005 for CIFAR-10, and 32 and 0.001 for CIFAR-100. We set the total number of training rounds as 100, 200, and 200 for these three datasets, respectively, indicating that Hier-FUN will terminate the *in-cluster training* at round of 100, 200, and 200 and enters the *unlearning and fine-tune* phase. Additionally, the local training iterations are specified as 100, 20, and 30 for three datasets. It's worth noting that the time required to finish one round varies significantly for different training methods. Consequently, we conduct a comparison of test accuracy, considering both the training time and the time needed to reach the specified accuracy. For Hier-FUN, the cosine similarity threshold δ for calculating correlation matrix is empirically set as 0.9, 0.2, and 0.1 for FMNIST, CIFAR-10, and CIFAR-100.

B. Experimental Results

1) Determination of Threshold δ: In order to find an appropriate value for cosine similarity threshold δ, we conduct an evaluation experiment on FMNIST with 21 edge devices. Concretely, the cosine similarity among gradients of any two devices under three Non-IID levels is plotted in Fig. 3, where the horizontal axis and vertical axis denote the device id. According to Fig. 3, we can see that the devices with similar data distribution yield a cosine similarity close to 1 even though the Non-IID level changes, e.g., devices 1-4 in Fig. 3(a)–(c). For the devices with different data distributions, the value of cosine similarity varies significantly with Non-IID levels. For instance, by Fig. 3(a), the cosine similarity value for devices with different data distributions is lower than 0.4. In comparison, in Fig. 3(c), the cosine similarity value between devices 6 and 10 reaches 0.81. We note that the data distributions of all devices become similar as the value of p decreases, and the cosine similarity value for devices in different partitions is increased. To divide devices into clusters according to their data distributions, the value of δ should be chosen from the range of (δ_d, δ_s), where δ_d represents the maximum cosine similarity among the devices with different data distributions, and δ_s denotes the minimum cosine similarity among the devices with similar data distributions. By Fig. 3, the candidate ranges are (0.39, 1.0), (0.43, 0.99) and (0.81, 0.99) for Level 1, Level 2 and Level 3, respectively. To this end, we specify the cosine similarity threshold δ for FMNIST as 0.9, so as to distinguish the variation of data distributions over FMNIST under three Non-IID levels. Using the same

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

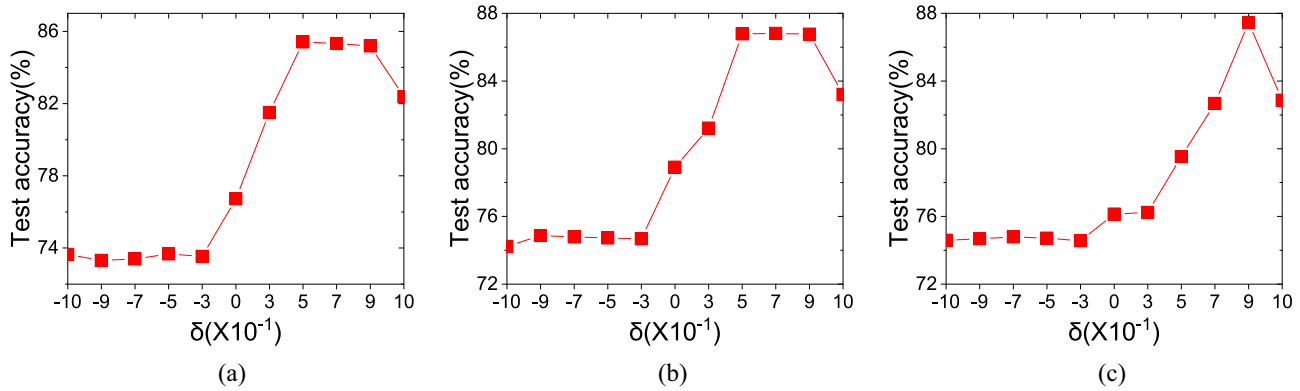


Fig. 4. Test accuracy of the models trained with different δ over FMNIST under three Non-IID levels. (a) Level 1. (b) Level 2. (c) Level 3.

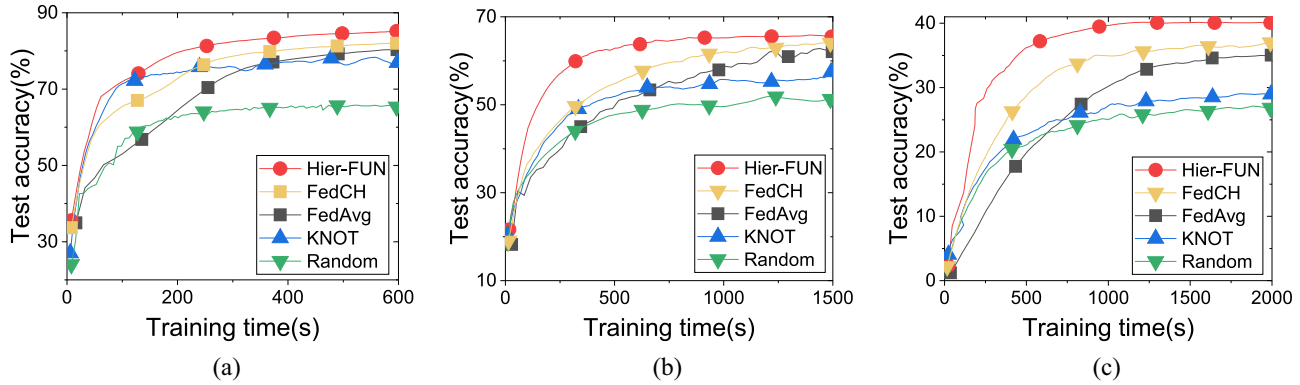


Fig. 5. Test accuracy curves of the models trained with different methods under the same training time. (a) MLP over FMNIST. (b) AlexNet over CIFAR-10. (c) ResNet-18 over CIFAR-100.

methodology, we set the values of δ as 0.2 and 0.1 for CIFAR-10 and CIFAR-100, respectively.

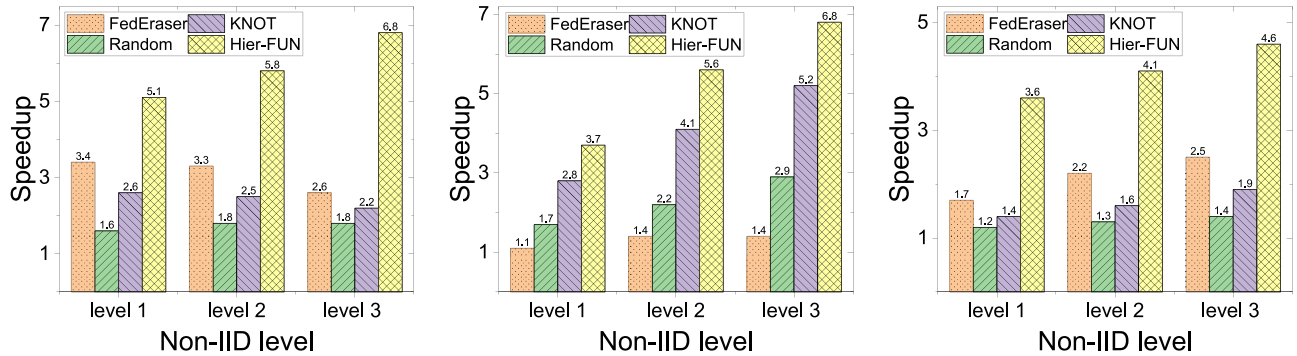
To evaluate the impact of varying the threshold δ, we conduct an extra experiment on FMNIST by adjusting δ across a range from -1.0 to 1.0 under three Non-IID levels. Specifically, when δ is set too small, each device serves as an independent cluster and trains its local model independently, leading to a diminished performance. For instance, Fig. 4(a) demonstrates that the test accuracy is around 73.51% when the value of δ is lower than -0.3. Conversely, if δ is excessively large (e.g., 1.0), all devices are consolidated into a single cluster, resulting in a training process similar to FedAvg. This consolidation can degrade Hier-FUN's performance due to the heterogeneous resources of edge devices. If δ is optimally configured (e.g., 0.9), Hier-FUN can distinguish the variation of data distributions among devices, and will guarantee the model performance. Consequently, we set δ as 0.9 for FMNIST in subsequent experiments based on these observations.

2) Overall Learning Performance: We conduct the second set of experiments to compare the overall learning performance of Hier-FUN and existing methods (i.e., FedCH, FedAvg, Random, and KNOT). Because ReTrain is only for unlearning and the learning trajectory of FedEraser is similar to that of FedAvg, they are not included. Specifically, the test accuracy curves of the models trained with different methods are illustrated in Fig. 5, where the horizontal axis represents the training time. By Fig. 5, we can observe that the test accuracy curves of all methods increase with training time. Among the baselines, FedCH always achieves higher test accuracy, since it assigns the devices with similar hardware specifications into the same cluster, which can reduce the waiting time and accelerate the learning process. Moreover, KNOT achieves higher test accuracy than FedAvg in the early stage of training, since the devices with similar hardware specifications are assigned to the same cluster and the aggregation is performed asynchronously. However, as the training progresses, the test accuracy of KNOT is lower than that of FedAvg. For example, by Fig. 5(c), when the training time equals 1,523s, the test accuracy of KNOT is 28.42%, while FedAvg obtains the test accuracy of 34.14%. The reason lies in that the clusters of KNOT are established by incorporating the devices with similar data distribution and lack data diversity. On the contrary, Hier-FUN tends to divide the devices with similar hardware specifications but complementary data distributions into the same cluster. Consequently, Hier-FUN always shows superior performance to baselines over different datasets. For instance, in Fig. 5(b), when the training time equals 505s, Hier-FUN achieves the test accuracy of 62.90% over CIFAR-10. In comparison, the test accuracy of other methods (i.e., FedCH FedAvg, KNOT, and Random) is 54.99%, 51.01%, 51.94%, and 47.86%. In other words, given the same training time, Hier-FUN outperforms the existing methods and can achieve higher test accuracy. Generally, over three datasets, Hier-FUN

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

TABLE II
TABLE OF TARGET ACCURACY AND RETENTION ACCURACY OF DIFFERENT UNLEARNING METHODS OVER THREE DATASETS

		FMNIST			CIFAR-10			CIFAR-100		
Methods	Metrics	*level 1*	*level 2*	*level 3*	*level 1*	*level 2*	*level 3*	*level 1*	*level 2*	*level 3*
ReTrain	target accuracy	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%
	retention accuracy	78.15%	79.48%	81.13%	68.27%	71.79%	74.61%	38.54%	39.23%	39.86%
FedEraser	target accuracy	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%
	retention accuracy	83.61%	84.51%	85.29%	69.41%	72.36%	75.86%	40.69%	41.64%	43.57%
KNOT	target accuracy	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%
	retention accuracy	82.48%	84.70%	85.62%	75.04%	77.84%	79.08%	39.80%	40.21%	42.60%
Random	target accuracy	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%
	retention accuracy	82.20%	84.20%	85.37%	72.44%	75.24%	78.62%	38.86%	40.63%	43.05%
Hier-FUN	target accuracy	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%	00.00%
	retention accuracy	**85.18%**	**86.23%**	**86.93%**	**78.68%**	**80.57%**	**83.49%**	**44.17%**	**45.72%**	**46.40%**



(a) (b) (c)

Fig. 6. Speedup of different methods compared with ReTrain when achieving the specified retention accuracy under three Non-IID levels. (a) MLP over FMNIST. (b) AlexNet over CIFAR-10. (c) ResNet-18 over CIFAR-100.

can improve the test accuracy by at least 3.19%, 1.43%, and 3.07%, indicating that Hier-FUN can accelerate the learning process on heterogeneous devices with Non-IID datasets.

3) Unlearning Performance: The third set of experiments is performed over three datasets to evaluate the unlearning performance of Hier-FUN under different Non-IID levels. Concretely, we compare the target accuracy and retention accuracy of different unlearning methods (i.e., ReTrain, FedEraser, KNOT, Random, and Hier-FUN), given the same time of 300s, 750s, and 1000s over FMNIST, CIFAR-10, and CIFAR-100, respectively. The results are presented in Table II, where target accuracy and retention accuracy represent the accuracy of unlearned model on target data and retained data, respectively. According to Table II, we can find that the target accuracy of all the unlearning methods is 00.00% over three datasets with different Non-IID levels. It is evident that the knowledge learned from the data of target device is totally unlearned, showing the unlearning performance of all methods. In addition, we also compare the retention accuracy of different methods to evaluate the performance of unlearned model. It is observed that the retention accuracy increases for all methods when decreasing the percentage of Non-IID data from 0.9 to 0.6. For instance, over CIFAR-10, Random obtains the improved accuracy from 72.44% (Level 1) to 78.62% (Level 3), since the training data approach IID pattern if p is small. Moreover, given the same training time, ReTrain also achieves the lowest retention accuracy among all methods over three datasets, since retraining the model from scratch is usually time-consuming. Importantly, compared with the other unlearning methods (i.e., FedEraser, KNOT, and Random), Hier-FUN persistently obtains higher retention accuracy under different Non-IID levels. Concretely, Hier-FUN improves the test accuracy by 2.90%, 2.78%, and 2.85% on average under three Non-IID levels, demonstrating the superiority of the proposed framework in tackling Non-IID data.

4) Speedup: To comprehensively assess the unlearning efficiency of our proposed framework, we record the time required to achieve specified retention accuracy of 0.8, 0.7, and 0.4 over three distinct datasets. Subsequently, we calculate the speedup achieved by various unlearning approaches, including FedEraser, Random, KNOT, and Hier-FUN, in comparison with the baseline method of fully retraining from scratch (i.e., ReTrain). The comprehensive findings are presented in Fig. 6, where the horizontal axis represents the Non-IID levels. According to Fig. 6, it is evident that all methods exhibit favorable speedup outcomes when compared to the ReTrain baseline. Specifically, with respect to FedEraser, a maximum speedup of 3.4×, 1.4×, and 2.5× is achieved over three datasets, underscoring the efficiency of this approach in expediting the unlearning process. Turning attention to the hierarchical FUN approaches, including Random, KNOT, and our proposed Hier-FUN, we observe that the unlearning process is significantly accelerated through the strategic restriction of the influence sphere surrounding the target

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

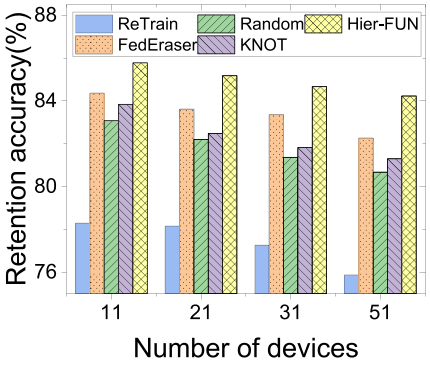


Fig. 7. Retention accuracy of the models trained with different methods on varying number of devices.

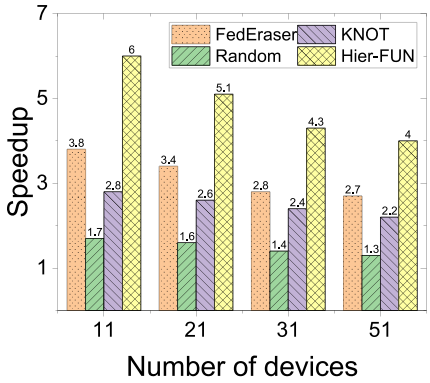


Fig. 8. Speedup of different methods compared with ReTrain on varying number of devices.

device. Consequently, the time required to attain the specified retention accuracy is remarkably reduced in comparison to the ReTrain approach. Notably, our proposed Hier-FUN framework consistently demonstrates superior speedup performance when compared to the other hierarchical FUN methods. In summary, over all three datasets, Hier-FUN exhibits speedup factors of 6.8×, 6.8×, and 4.6× at maximum, highlighting its efficiency in accelerating the unlearning process while maintaining retention accuracy.

5) Varying the Number of Devices: In consideration of the intricate model architectures inherent in AlexNet and ResNet-18, we embark on a comprehensive series of scalability experiments utilizing the MLP model over FMNIST. The objective of these experiments is to evaluate the performance of various FUN approaches by varying the number of devices from 11 to 51. The attained retention accuracy of different approaches, constrained within a fixed time budget of 300 s, is depicted in Fig. 7, where the horizontal axis signifies the varying number of devices employed in the environmental setup. By Fig. 7, a discernible trend becomes apparent: the retention accuracy of all methods experiences a notable decline as the number of edge devices increases. This decline can be attributed to the diminution of the training dataset available to each device as the total number of devices expands, inevitably compromising the overall generalization performance of the trained models. Among the various approaches evaluated, it is evident that ReTrain consistently yields the lowest retention accuracy within the same training time. This observation is in line with expectations, as the process of retraining the model from scratch incurs a substantial time overhead. In stark contrast, FedEraser consistently outperforms all other methods (except for Hier-FUN). This trend is consistent with the findings presented in Table II and Fig. 6, affirming the efficacy of FedEraser in achieving accelerated unlearning. Notably, Hier-FUN emerges as the framework consistently achieving the highest retention accuracy across all experimental settings. On average, Hier-FUN demonstrates an improvement in retention accuracy of 2.09% across varying numbers of edge devices, underscoring its effectiveness in mitigating the adverse effects of increasing device count on retention accuracy and solidifying its position as a promising approach for scalable and efficient FUN implementations.

Furthermore, we extend our analysis to evaluate the speedup achieved by various unlearning approaches, including FedEraser, Random, KNOT, and Hier-FUN, in comparison to the baseline method of fully retraining from scratch (i.e., ReTrain). We focus specifically on achieving a retention accuracy of 0.8 over the FMNIST dataset, with the varying number of participating devices from 11 to 51. The outcomes of these speedup evaluations are depicted in Fig. 8, where the horizontal axis denotes the number of devices employed in the experimental setting. As illustrated by Fig. 8, it is evident that the speedup exhibited by each method experiences a degradation when the number of devices is increased, mirroring the observed trend of retention accuracy as depicted in Fig. 7. For FedEraser, competitive speedup performance is consistently achieved across varying number of devices, positioning it favorably amongst other FUN approaches except for Hier-FUN. Among the hierarchical approaches, namely, Random, KNOT, and Hier-FUN, Random consistently demonstrates the lowest speedup. For instance, when training with 51 devices, Random achieves a speedup of 1.3×, while KNOT and Hier-FUN obtain a speedup of 2.2× and 4.0×, respectively. Despite a slight decline observed in the trend, Hier-FUN always outperforms all other baselines in terms of speedup. Specifically, with the varying number of devices, Hier-FUN achieves a maximum speedup of 6.0× over the FMNIST dataset, underscoring its effectiveness in expediting the unlearning process.

6) Varying the Resource Budgets: In EC, the participating devices in FL are various kinds of mobile devices, which are mostly powered by batteries. However, considering the constrained battery capacity of devices, the running time and energy usage for model training are also restricted. To this end, we aim to evaluate the impact of constrained resources on Hier-FUN by comparing the retention accuracy of different methods under various battery resource (running time) budgets. The results are depicted in Fig. 9, where the horizontal axis denotes the resource budget. According to Fig. 9, as the resource budgets increase, the retention accuracy of all the methods is improved. Nevertheless, given the same resource budget, Hier-FUN always achieves higher retention accuracy than the baseline methods. Concretely, by Fig. 9(b), when the resource (time) budget is 400s, Hier-FUN

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

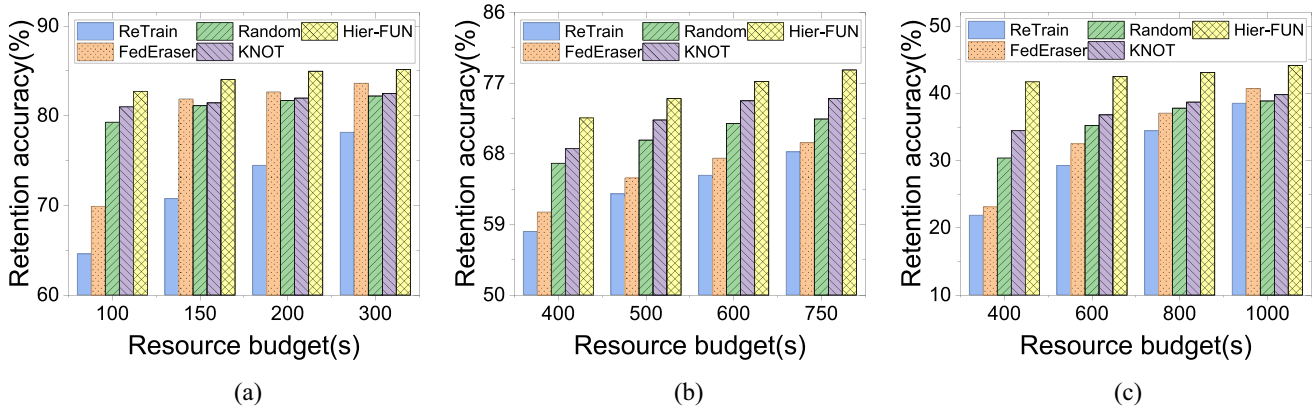


Fig. 9. Retention accuracy of the models trained with different methods under various resource budgets. (a) MLP over FMNIST. (b) AlexNet over CIFAR-10. (c) ResNet-18 over CIFAR-100.

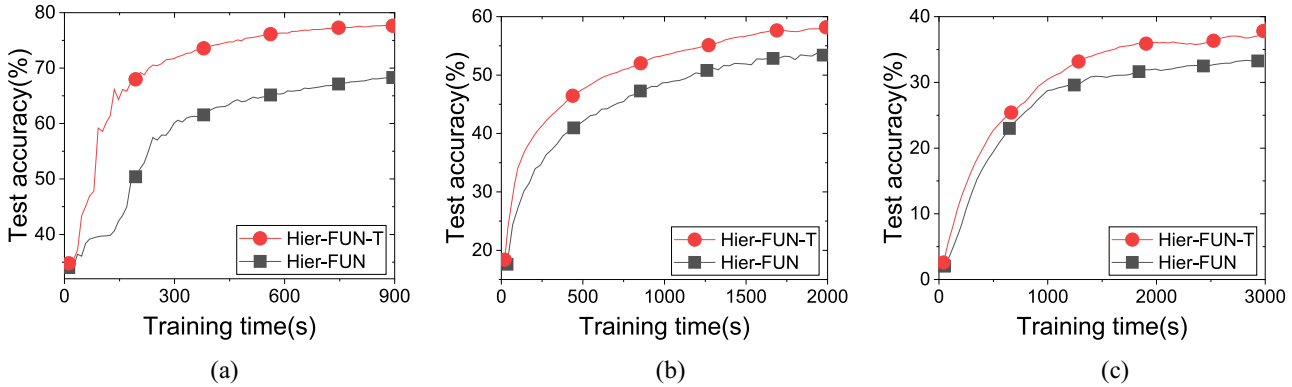


Fig. 10. Test accuracy curves of the models trained with Hier-FUN and Hier-FUN-T on three datasets. (a) MLP over FMNIST. (b) Alexnet over CIFAR-10. (c) ResNet-18 over CIFAR-100.

obtains the retention accuracy of 72.59% on CIFAR-10. In comparison, the retention accuracy of baselines (i.e., ReTrain, FedEraser, Random, and KNOT) is 58.11%, 60.56%, 66.79%, and 68.68%. Overall, given the time budgets of 300s, 750s, and 1000s over three datasets, Hier-FUN improves the retention accuracy at least by 1.57%, 3.64%, and 3.48% compared with the baselines, which demonstrates the effectiveness of the proposed framework.

7) Impact of Communication Timeout: To evaluate the impact of communication timeout issue, we compare the performance of Hier-FUN and Hier-FUN-T under poor network condition. The training time is configured as 900s, 2000s, and 3000s for FMNIST, CIFAR-10, and CIFAR-100, respectively. Concretely, the test accuracy of Hier-FUN and Hier-FUN-T is compared and depicted in Fig. 10. By Fig. 10, it is obvious that both Hier-FUN and Hier-FUN-T obtain the increased test accuracy. Nevertheless, the test accuracy of Hier-FUN-T is always higher than that of Hier-FUN given the same training time. For instance, according to Fig. 10(b), when the training time reaches 990s, the test accuracy of Hier-FUN is 48.67% on CIFAR-10, while Hier-FUN-T obtains the test accuracy of 53.36%. For Hier-FUN, the server must remain in a state of anticipation for the devices that are encountering communication timeout, and the training time required for the model to reach convergence is increased. Instead, Hier-FUN-T retransmits the model if no reply is received from the server within the RTO, which alleviates the communication timeout issue to some degree. Overall, the test accuracy achieved by Hier-FUN-T is approximately 6.70% higher than that of Hier-FUN over three datasets, demonstrating the efficacy of timeout retransmission mechanism.

V. Related Work

A. Hierarchical Federated Learning

In recent years, substantial progress has been achieved in the integration of FL with beyond 5G (B5G) edge networks [9], [10], [13], [14], [15], [53], [54]. For instance, Xu et al. [10] introduced a learning rate optimization method that utilizes over-the-air computation to enhance the performance of FL in wireless communication networks, thereby providing valuable insights into the adaptation of FL under dynamic network conditions. Concurrently, Chen et al. [13] developed a joint learning and communication framework that facilitates the efficient operation of FL over wireless networks, aiming to improve the synergy between learning processes and data transmission. Moreover, Cui et al. [14] applied multiagent reinforcement learning to formulate a cooperative multitype task offloading strategy for the Internet of Vehicles within B5G/6G networks, demonstrating the flexibility and efficiency of FL in complex mobile environments. Regarding energy efficiency, Yang et al. [15] examined strategies for

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

achieving energy-efficient FL over wireless communication networks, proposing optimization algorithms designed to minimize both communication overhead and energy consumption. Furthermore, Ni et al. [54] investigated the potential of integrating over-the-air FL with nonorthogonal multiple access, employing reconfigurable intelligent surfaces to advance FL applications in future networks. However, the aforementioned studies predominantly focus on a flat architecture, which may lead to an increased communication load as all device updates are transmitted directly to a central server.

To address the communication bottleneck on central servers, numerous research endeavors have emerged in HFL [55], [56], [57], [58], [59]. Initially, foundational frameworks, such as those proposed by Liu et al. [55], introduce device-edge-cloud FL architectures aimed at alleviating communication overhead on central servers and expediting convergence rate. However, these early frameworks, while effective in principle, often overlook the inherent heterogeneity of resources among edge devices, potentially impeding training efficiency, particularly in scenarios involving a large number of participating devices. Consequently, recent works have shifted their focus toward mitigating resource constraints within the paradigm of HFL [19], [41], [57], [60], [61], [62]. For instance, Luo et al. [57] introduced strategies for resource allocation on edge devices, optimizing training performance while accounting for energy consumption and latency constraints within EC scenarios. Similarly, Tu et al. [61] endeavored to determine the optimal allocation of data processing responsibilities among edge servers and devices. Nevertheless, this approach raises privacy concerns regarding the sensitive data on edge devices. Wang et al. [41] proposed a resource-efficient hierarchical framework for FL, named RFL-HA, which leverages hierarchical aggregation to reduce the completion time required for model training. Although resource heterogeneity can be addressed by asynchronous intercluster aggregation, RFL-HA is not always effective in tackling Non-IID data, potentially leading to a performance degradation of the global model. Additionally, the aforementioned methods are proposed to improve the training efficiency of FL with hierarchical architectures and cannot achieve the unlearning of specific devices.

B. Federated Unlearning

FUN has emerged as a crucial mechanism aimed at enhancing data privacy through the selective exclusion of contributions from specific devices [23], [25], [26], [28], [39]. For instance, FedEraser, as proposed by Liu et al., achieves this objective by calibrating the historical gradients stored on the server, thereby eliminating a device's contribution [39]. Similarly, Wu et al. [23] also achieve the unlearning using historical gradients on the server, and decrease the success rate of backdoor attack utilizing knowledge distillation. However, the storage of historical gradients at each round across all devices imposes significant memory overhead on the server, particularly in environments with a large number of devices in EC. To reduce the memory overhead on the server, Halimi et al. [26] proposed a novel approach whereby the target data is unlearned through the reversal of the learning process. This involves training the model to maximize the local empirical loss of the target device, effectively removing its contribution. Nevertheless, regarding the Non-IID data, the performance of global model may be compromised when gradient ascent is performed on the target device. Wang et al. [25] presented an innovative strategy wherein they observe the varying contribution of different channels to various categories. Through the pruning of the most discriminative channels, they achieve the removal of the target class. However, it's important to note that this approach is limited to the removal of specific classes and is tailored exclusively for CNNs.

The aforementioned methods primarily focus on accelerating the unlearning process but fail to consider the adverse effects of device heterogeneity on model training. To this end, Su and Li [28] proposed KNOT, a novel approach that integrates asynchronous updates and hierarchical architectures into the FUN framework. Specifically, KNOT performs clustering on devices based on their training time and data distributions, and aggregates the model updates within the confines of each cluster solely, which can constrain the influence sphere of target device and expedite the unlearning process. Nevertheless, despite its merits, KNOT exhibits certain limitations. For instance, the clustering process in KNOT tends to group devices with similar data distributions together. While this approach facilitates efficient model training, it may compromise the generalization performance of the model, particularly on Non-IID data [63]. Furthermore, the requirement to specify the number of clusters in advance necessitates extensive prior knowledge, which can be impractical in real-world scenarios.

Diverging from the approaches delineated above, we introduce a pioneering hierarchical framework tailored specifically for FUN, denoted as Hier-FUN. The overarching objective of Hier-FUN is to address the trifecta of challenges posed by *constrained resources*, *heterogeneous devices*, and *Non-IID data* concurrently. It is imperative to emphasize that Hier-FUN operates independently of existing unlearning techniques, thus offering a complementary approach to addressing the aforementioned challenges. Furthermore, Hier-FUN can seamlessly integrate with pre-existing unlearning algorithms, thereby presenting opportunities for synergistic enhancements in performance and efficacy. This strategic flexibility underscores the versatility and adaptability of Hier-FUN in heterogeneous EC scenarios.

VI. Conclusion

This article delves into addressing the challenges posed by heterogeneous devices and Non-IID data distributions within the context of FL and Unlearning. Our focus revolves around leveraging hierarchical clustering techniques to expedite both the learning and unlearning processes. Concretely, we introduce Hier-FUN, a novel framework meticulously designed to automatically determine an appropriate cluster number based on the available resources and data distributions of participating devices. Subsequently, we allocate the devices with similar hardware specifications and complementary data distributions to the same cluster. This strategic clustering

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

approach is integrated with in-cluster training, thereby constraining the influence sphere of target devices and accelerating the unlearning process effectively. Our proposed framework underwent rigorous evaluation through extensive experiments conducted over three real-world datasets. The experimental findings unequivocally demonstrate the efficiency of Hier-FUN in addressing the challenges associated with heterogeneous devices and Non-IID data in EC scenarios.

REFERENCES

[1] B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," in *Proc. Artif. Intell. Statist.*, 2017, pp. 1273–1282.

[2] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet Things J.*, vol. 3, no. 5, pp. 637–646, Oct. 2016.

[3] A. Hard et al., "Federated learning for mobile keyboard prediction," 2018, *arXiv:1811.03604*.

[4] Z. Xiao, X. Xu, H. Xing, F. Song, X. Wang, and B. Zhao, "A federated learning system with enhanced feature extraction for human activity recognition," *Knowl.-Based Syst.*, vol. 229, Oct. 2021, Art. no. 107338.

[5] Y. Yin, Y. Li, H. Gao, T. Liang, and Q. Pan, "FGC: GCN-based federated learning approach for trust industrial service recommendation," *IEEE Trans. Ind. Informat.*, vol. 19, no. 3, pp. 3240–3250, Mar. 2023.

[6] R. S. Antunes, C. André da Costa, A. Küderle, I. A. Yari, and B. Eskofier, "Federated learning for healthcare: Systematic review and architecture proposal," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 4, pp. 1–23, 2022.

[7] Y. Qiao et al., "Mp-FedCL: Multiprototype federated contrastive learning for edge intelligence," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8604–8623, Mar. 2024.

[8] G. Li, J. Cai, C. He, X. Zhang, and H. Chen, "Online incentive mechanism designs for asynchronous federated learning in edge computing," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 7787–7804, Mar. 2024.

[9] W. Xu, Z. Yang, D. W. K. Ng, M. Levorato, Y. C. Eldar, and M. Debbah, "Edge learning for B5G networks with distributed signal processing: Semantic communication, edge computing, and wireless sensing," *IEEE J. Sel. Topics Signal Process.*, vol. 17, no. 1, pp. 9–39, Jan. 2023.

[10] C. Xu, S. Liu, Z. Yang, Y. Huang, and K.-K. Wong, "Learning rate optimization for federated learning exploiting over-the-air computation," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 12, pp. 3742–3756, Dec. 2021.

[11] F. Jiang, K. Wang, L. Dong, C. Pan, W. Xu, and K. Yang, "Deep-learning-based joint resource scheduling algorithms for hybrid MEC networks," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6252–6265, Jul. 2020.

[12] J. Zhang, N. Li, and M. Dedeoglu, "Federated learning over wireless networks: A band-limited coordinated descent approach," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2021, pp. 1–10.

[13] M. Chen, Z. Yang, W. Saad, C. Yin, H. V. Poor, and S. Cui, "A joint learning and communications framework for federated learning over wireless networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 1, pp. 269–283, Jan. 2021.

[14] Y. Cui, H. Li, D. Zhang, A. Zhu, Y. Li, and H. Qiang, "Multiagent reinforcement learning-based cooperative multitype task offloading strategy for Internet of Vehicles in B5G/6G network," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12248–12260, Jul. 2023.

[15] Z. Yang, M. Chen, W. Saad, C. S. Hong, and M. Shikh-Bahaei, "Energy efficient federated learning over wireless communication networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 3, pp. 1935–1949, Mar. 2021.

[16] S. Chen, Z. Ma, and Z. Wang, "Accelerating hierarchical federated learning with adaptive aggregation frequency in edge computing," in *Proc. 4th Int. Conf. Comput., Netw. Internet Things*, 2023, pp. 990–995.

[17] Y. Xu, Z. Ma, H. Xu, S. Chen, J. Liu, and Y. Xue, "FedLC: Accelerating asynchronous federated learning in edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 5, pp. 5327–5343, May 2024.

[18] L. Wang, Y. Xu, H. Xu, M. Chen, and L. Huang, "Accelerating decentralized federated learning in heterogeneous edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 9, pp. 5001–5016, Sep. 2023.

[19] Z. Wang, H. Xu, J. Liu, Y. Xu, H. Huang, and Y. Zhao, "Accelerating federated learning with cluster construction and hierarchical aggregation," *IEEE Trans. Mobile Comput.*, vol. 22, no. 7, pp. 3805–3822, Jul. 2023.

[20] Z. Jiang, Y. Xu, H. Xu, L. Wang, C. Qiao, and L. Huang, "Joint model pruning and topology construction for accelerating decentralized machine learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 10, pp. 2827–2842, Oct. 2023.

[21] M. Mesarčík and O. Hamuľák, "General data protection regulation: Current challenges and future directions," in *E-Governance in the European Union: Strategies, Tools, and Implementation*. Cham, Switzerland: Springer Nat., 2024, pp. 117–133, doi: 10.1007/978-3-031-56045-3_9.

[22] E. Illman and P. Temple, "California consumer privacy act," *Bus. Lawyer*, vol. 75, no. 1, pp. 1637–1646, 2019.

[23] C. Wu, S. Zhu, and P. Mitra, "Federated unlearning with knowledge distillation," 2022, *arXiv:2201.09441*.

[24] M. Li et al., "Scaling distributed machine learning with the parameter server," in *Proc. 11th USENIX Symp. Oper. Syst. Design Implement. (OSDI)*, 2014, pp. 583–598.

[25] J. Wang, S. Guo, X. Xie, and H. Qi, "Federated unlearning via class-discriminative pruning," in *Proc. ACM Web Conf.*, 2022, pp. 622–632.

[26] A. Halimi, S. Kadhe, A. Rawat, and N. Baracaldo, "Federated unlearning: How to efficiently erase a client in FL?" 2022, *arXiv:2207.05521*.

[27] Y. Liu, L. Xu, X. Yuan, C. Wang, and B. Li, "The right to be forgotten in federated learning: An efficient realization with rapid retraining," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2022, pp. 1749–1758.

[28] N. Su and B. Li, "Asynchronous federated unlearning," in *Proc. IEEE INFOCOM*, 2023, pp. 1–10.

[29] Y. Zhao, P. Wang, H. Qi, J. Huang, Z. Wei, and Q. Zhang, "Federated unlearning with momentum degradation," *IEEE Internet Things J.*, vol. 11, no. 5, pp. 8860–8870, Mar. 2024.

[30] X. Fang and M. Ye, "Robust federated learning with noisy and heterogeneous clients," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10072–10081.

[31] M. Tang et al., "FedCor: Correlation-based active client selection strategy for heterogeneous federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10102–10111.

[32] Z. Ma, Y. Xu, H. Xu, Z. Meng, L. Huang, and Y. Xue, "Adaptive batch size for federated learning in resource-constrained edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 1, pp. 37–53, Jan. 2023.

[33] F. Dou, J. Lu, T. Zhu, and J. Bi, "On-device indoor positioning: A federated reinforcement learning approach with heterogeneous devices," *IEEE Internet Things J.*, vol. 11, no. 3, pp. 3909–3926, Feb. 2024.

[34] P. Pene, W. Liao, and W. Yu, "Incentive design for heterogeneous client selection: A robust federated learning approach," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 5939–5950, Feb. 2024.

[35] Z. Ma, Y. Xu, H. Xu, J. Liu, and Y. Xue, "Like attracts like: Personalized federated learning in Decentralized edge computing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 2, pp. 1080–1096, Feb. 2024.

[36] W. Huang, M. Ye, and B. Du, "Learn from others and be yourself in heterogeneous federated learning," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2022, pp. 10143–10153.

[37] B. Li, M. N. Schmidt, T. S. Alstrøm, and S. U. Stich, "On the effectiveness of partial variance reduction in federated learning with heterogeneous data," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2023, pp. 3964–3973.

[38] J. Shu et al., "Clustered federated multitask learning on non-IID data with enhanced privacy," *IEEE Internet Things J.*, vol. 10, no. 4, pp. 3453–3467, Feb. 2023.

[39] G. Liu, X. Ma, Y. Yang, C. Wang, and J. Liu, "FedEraser: Enabling efficient client-level data removal from federated learning models," in *Proc. IEEE/ACM 29th Int. Symp. Qual. Service (IWQOS)*, 2021, pp. 1–10.

[40] A. Golatkar, A. Achille, A. Ravichandran, M. Polito, and S. Soatto, "Mixed-privacy forgetting in deep networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2021, pp. 792–801.

[41] Z. Wang, H. Xu, J. Liu, H. Huang, C. Qiao, and Y. Zhao, "Resource-efficient federated learning with hierarchical aggregation in edge computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2021, pp. 1–10.

[42] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA, USA: MIT Press, 2022.

[43] A. L. Dulmage and N. S. Mendelsohn, "Coverings of bipartite graphs," *Can. J. Math.*, vol. 10, pp. 517–534, Jan. 1958.

[44] H. Zha, X. He, C. Ding, H. Simon, and M. Gu, "Bipartite graph partitioning and data clustering," in *Proc. 10th Int. Conf. Inf. Knowl. Manag.*, 2001, pp. 25–32.

[45] H. W. Kuhn, "The Hungarian method for the assignment problem," *Nav. Res. Logist. Quart.*, vol. 2, nos. 1–2, pp. 83–97, 1955.

[46] X. Liu, "Generic face alignment using boosted appearance model," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2007, pp. 1–8.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

[47] V. Paxson, M. Allman, J. Chu, and M. Sargent, "Computing TCP's retransmission timer," Internet Eng. Task Force, RFC 6298, 2011.
[48] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. 33rd Int. Conf. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 8026–8037.
[49] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," 2017, *arXiv:1708.07747*.
[50] A. Pinkus, "Approximation theory of the MLP model in neural networks," *Acta Numerica*, vol. 8, pp. 143–195, Jan. 1999.
[51] S. Li, L. Wang, J. Li, and Y. Yao "Image classification algorithm based on improved AlexNet," *J. Phys., Conf. Ser.*, vol. 1813, no. 1, 2021, Art. no. 12051. [Online]. Available: https://iopscience.iop.org/article/10.1088/1742-6596/1813/1/012051/meta
[52] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. 14th Eur. Conf. Comput. Vis. (ECCV)*, Amsterdam, Netherlands, 2016, pp. 630–645.
[53] J. Yao, W. Xu, Z. Yang, X. You, M. Bennis, and H. V. Poor, "Wireless federated learning over resource-constrained networks: Digital versus analog transmissions," *IEEE Trans. Wireless Commun.*, vol. 23, no. 10, pp. 14020–14036, Oct. 2024.
[54] W. Ni, Y. Liu, Z. Yang, and H. Tian, "Over-the-air federated learning and non-orthogonal multiple access unified by reconfigurable intelligent surface," in *Proc. INFOCOM IEEE Conf. Comput. Commun. Workshops (INFOCOM WKSHPS)*, 2021, pp. 1–6.
[55] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Client-edge-cloud hierarchical federated learning," in *Proc. IEEE Int. Conf. Commun. (ICC)*, 2020, pp. 1–6.
[56] D. Liu, X. Chen, Z. Zhou, and Q. Ling, "HierTrain: Fast hierarchical edge AI learning with hybrid parallelism in mobile-edge-cloud computing," *IEEE Open J. Commun. Soc.*, vol. 1, pp. 634–645, 2020.
[57] S. Luo, X. Chen, Q. Wu, Z. Zhou, and S. Yu, "HFEL: Joint edge association and resource allocation for cost-efficient hierarchical federated edge learning," *IEEE Trans. Wireless Commun.*, vol. 19, no. 10, pp. 6535–6548, Oct. 2020.
[58] L. Liu, J. Zhang, S. Song, and K. B. Letaief, "Hierarchical quantized federated learning: Convergence analysis and system design," 2021, *arXiv:2103.14272*.
[59] L. Su, R. Zhou, N. Wang, J. Chen, and Z. Li, "Low-latency hierarchical federated learning in wireless edge networks," *IEEE Internet Things J.*, vol. 11, no. 4, pp. 6943–6960, Feb. 2024.
[60] M. S. H. Abad, E. Ozfatura, D. Gündüz, and O. Ercetin, "Hierarchical federated learning across heterogeneous cellular networks," in *Proc. IEEE Int. Conf. Acoust., Speech Signal Process. (ICASSP)*, 2020, pp. 8866–8870.
[61] Y. Tu, Y. Ruan, S. Wagle, C. G. Brinton, and C. Joe-Wong, "Network-aware optimization of distributed learning for fog computing," in *Proc. IEEE INFOCOM Conf. Comput. Commun.*, 2020, pp. 2509–2518.
[62] X. Huang, Y. Wu, C. Liang, Q. Chen, and J. Zhang, "Distance-aware hierarchical federated learning in blockchain-enabled edge computing network," *IEEE Internet Things J.*, vol. 10, no. 21, pp. 19163–19176, Nov. 2023.
[63] X. Ma, J. Zhu, Z. Lin, S. Chen, and Y. Qin, "A state-of-the-art survey on solving non-IID data in federated learning," *Future Gener. Comput. Syst.*, vol. 135, pp. 244–258, Oct. 2022.

Zhenguo Ma received the B.S. degree in software engineering from Shandong University, Jinan, China, in 2018, and the Ph.D. degree from the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China, in 2023.

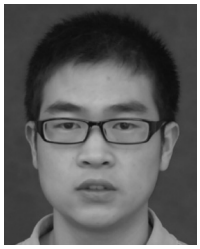
He is currently working with the Research Center for Data Hub and Security, Zhejiang Laboratory, Hangzhou, China. He has published more than ten papers in famous journal and conferences, including IEEE TRANSACTIONS ON MOBILE COMPUTING, *Computer Networks*, International Symposium on Cluster Cloud and Internet Computing, and International Conference on Computing Networks and Internet of Things. His research interests include edge computing, federated learning, and unlearning.

Huaqing Tu received the B.S. degree in computer science from Northeastern University, Shenyang, China, in 2018, and the Ph.D. degree in computer software and theory from the University of Science and Technology of China, Hefei, China, in 2023.

She is currently a Postdoctoral Fellow with Zhejiang Laboratory, Hangzhou, China. She has published more than ten articles in famous journals and conferences, including IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, *Computer Networks*, and IEEE/ACM International Symposium on Quality of Service. Her main research interests are software-defined networks, programmable networks, and cloud computing.

Li Zhou received the Ph.D. degree in statistics from the University of Le Mans, Le Mans, France, in 2013.

She was a Lecturer with the School of Mathematics and Statistics, Shandong University, Jinan, China, from 2013 to 2022, and she jointed subsequently with Zhejiang Laboratory, Hangzhou, China, in 2022. Her research interests is in probability and statistics, both on theory and applications, including statistics for diffusion process, privacy protecting, machine learning and their applications in finance, smart transportation, and data generation.



Pengli Ji received the Ph.D. degree in computer science from Huazhong University of Science and Technology, Wuhan, China, in 2017.

From 2017 to 2019, he was a Senior Engineer with Huawei, Shenzhen, China. From 2019 to 2021, he joined Hangzhou Dianzi University, Hangzhou, China, as an Assistant Professor. From 2021 to 2022, he worked with Alibaba Group, Hangzhou, as a Senior Engineer. Since 2022, he has been working with the Research Center for Data Hub and Security, Zhejiang Laboratory, Hangzhou. His current research interests include operations research, optimization algorithms, reinforcement learning, large language model, and federated learning.

Xiaoran Yan (Member, IEEE) received the B.S. degree in computer science from Zhejiang University, Hangzhou, China, in 2007, and the Ph.D. degree in computer science from the University of New Mexico, Albuquerque, NM, USA, in 2013.

From 2013 to 2015, he was a Postdoctoral Research Associate with the Information Sciences Institute, University of Southern California, Los Angeles, CA, USA. From 2015 to 2021, he was an Assistant Research Scientist with the Indiana University Network Science Institute, Bloomington, IN, USA. In 2021, he joined the Research Institute of Artificial Intelligence, Zhejiang Laboratory, Hangzhou. He has co-directed NIH and IMLS Projects, authored multiple journal and conference papers, and has served on program committees for major organizations, such as IEEE and AAAI. His research interests include network science, statistical machine learning, data privacy, and their trans-disciplinary applications in social, communications and biomedical networks.

Hongli Xu (Member, IEEE) received the B.S. degree in computer science and the Ph.D. degree in computer software and theory from the University of Science and Technology of China (USTC), Hefei, China, in 2002 and 2007, respectively.

He is a Professor with the School of Computer Science and Technology, USTC. He has published more than 100 papers in famous journals and conferences, including the IEEE/ACM TRANSACTIONS ON NETWORKING, IEEE TRANSACTIONS ON MOBILE COMPUTING, IEEE TRANSACTIONS ON PARALLEL AND DISTRIBUTED SYSTEMS, Infocom, and ICNP. He has also held more than 30 patents. His main research interest is software defined networks, edge computing, and Internet of Thing.

Prof. Xu was awarded the Outstanding Youth Science Foundation of NSFC in 2018. He has won the Best Paper Award or the Best Paper Candidate in several famous conferences.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.

Zhiyuan Wang received the B.S. degree from Jilin University, Changchun, China, in 2019. He is currently pursuing the Ph.D. degree with the School of Computer Science, University of Science and Technology of China, Hefei, China.

His main research interests are edge computing, artificial intelligence, federated learning, and distributed machine learning.

Suo Chen received the B.S. degree in information and computing sciences from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2018. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, University of Science and Technology of China, Hefei, China.

His research interests include edge computing and federated learning.

Authorized licensed use limited to: Zhejiang Lab. Downloaded on August 03,2025 at 03:31:01 UTC from IEEE Xplore. Restrictions apply.